今天学习总结：

1.观察了整个流程tilelang怎么跑通代码，配置环境，最后成功跑通tilelang的测试。

2.DSL代码阅读

1.Tilelang实战环节

首先是配置环境，命令如下

```
git submodule update --init --recursive      # 3rdparty 很大；clone 漏 --recursive 这里就是空的
# ① 若机器没有预编译的 bishengir：先自编译 AscendNPU-IR（MLIR+C++ 大工程，几十分钟）
#    按 AscendNPU-IR 文档 cmake/ninja 构建，产物在 build/bin/bishengir-compile；记 commit（我们当时 19.1.7 / 31f6903）
# ② 编 TileLang 自身（C++ 绑定 + Python 包），复用上面编好的编译器，别重编一套几 G 的
bash install_npuir.sh --bishengir-path <AscendNPU-IR>/build
# ③ 已知坑：链接报 -pthread 相关错误 → 补 -pthread 后重跑
python -c "import tilelang; print(tilelang.__version__)"   # 通过，并记 VERSION
# ④ import 失败先查：bishengir-compile 是否在 PATH、LD_LIBRARY_PATH 是否指向 build/lib
```

这样跑通了： TileLang 代码 → 编译成 NPU 可执行产物 → 在 npu:0 上真实执行 → 数值和 torch 对齐

L2 看到 4 个信号：

1. `AscendNPU IR compile success` —— **编译器链路通了**
2. `Precompiling NPU launcher header..` —— 生成 host 侧启动器
3. 输出 `device='npu:0'`，不是 `cpu` —— 防静默 fallback
4. `All check passed` —— 精度对齐 torch

```
$ which bishengir-compile
/home/developer/Ascend/cann-9.0.0/bin/bishengir-compile

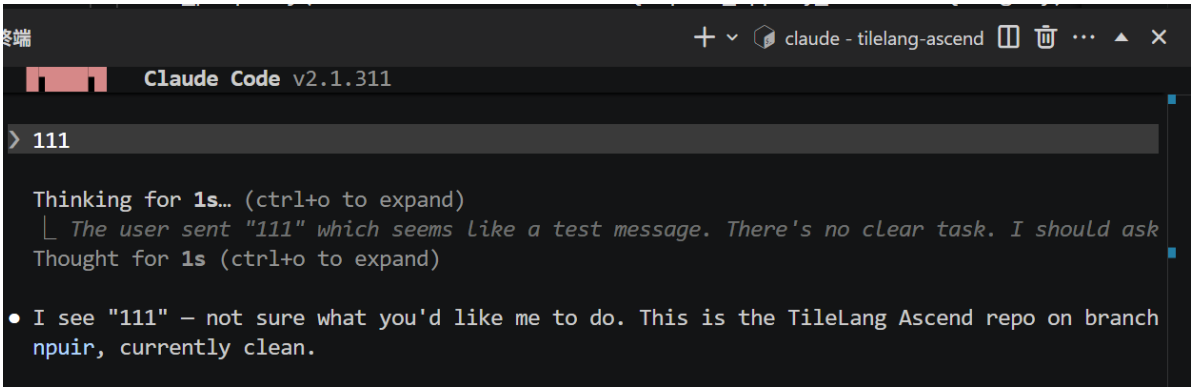
$ export TILELANG_CACHE_DIR=$(mktemp -d)

$ python examples/elementwise/vec_add_1d.py
AscendNPU IR compile success:
Precompiling NPU launcher header...
tensor([ 0.1419, -0.6733,  2.1362,  ..., -2.2451, -0.1236, -1.4902],
       device='npu:0')
tensor([ 0.1419, -0.6733,  2.1362,  ..., -2.2451, -0.1236, -1.4902],
       device='npu:0')
All check passed

exit=0

$ python -m pytest testing/npuir/active_ops/test_relu.py -s -q
AscendNPU IR compile success:
Precompiling NPU launcher header...
.AscendNPU IR compile success:
.AscendNPU IR compile success:
.AscendNPU IR compile success:
4 passed, 1 warning in 63.04s (0:01:03)
```

给CANN环境配置了一个claude code，用的ds的API，这样debug代码快一些



2.阅读DSL代码

我同时补齐了一些INFRA的相关知识，了解到执行模型，内存模型，数据搬运，并行与流水，计算原语，编译产物。这些是比较重要的。所以根据我的补齐，下面了解一下DSL的代码。

2.1 内存层次与核绑定

文档给出的容量口径：UB 通常 192KB，Vector 核私有；L1 通常 512KB（A2 系列），Cube 核私有；L0C 是 Cube 的累加器。GM 容量大但延迟高，所以需要 tiling。

这里有个贯穿全文的前提：`TILELANG_ASCEND_MODE` **不设就是 Expert**（`engine/lower.py:170-179`、`jit/jit_npu.py:1486-1492`），此时 `bishengir-compile` 会多收一个 `--disable-hivm-tensor-compile=true`；只有显式设成其他值（如 `Developer`）才走 tensor 路径。

分配接口按控制层级分两类：

- **显式空间（Expert）**：`alloc_ub` 落 UB（→ Vector 核），`alloc_L1` 落 L1（→ Cube 核），`alloc_L0C` 落 L0C（→ Cube 核），后两者通常配合 `T.Scope("Cube")` 使用。
- **共享空间**：`alloc_shared` 的默认 scope 是 `shared.dyn`（`language/allocate.py:22`）。Expert 下 codegen 把它翻成 `cbuf` 即 L1，并按 `AIC` 分配

（`src/target/codegen_npuir.cc:95-96、1338-1340`）——**落点是确定的。**只有 Developer 的 tensor 路径才把它当抽象 shared，按消费者是 Vector 还是 Cube 决定映射到 UB 还是 L1。
- `alloc_fragment`：默认 scope 是 `local.fragment`，没有独立文档条目，只在 GEMM 和 Parallel 示例里作为累加/片段缓冲出现。

API	默认 scope	Expert 落点	核
`alloc_ub`	`shared`	UB	AIV（Vector）
`alloc_shared`	`shared.dyn`	L1（`cbuf`）	AIC（Cube）
`alloc_L1`	`shared.dyn`	L1	AIC
`alloc_L0C`	`wmma.accumulator`	L0C（`cc`）	AIC
`alloc_fragment`	`local.fragment`	—	—

因此"选内存空间等于选计算单元"是成立的，而且 Expert 下没有模糊空间。

由此有一个实用推论：`alloc_shared` **+ 向量算子（**`T.vabs` **/** `T.vadd`**）是危险组合**——向量指令会被分到 Cube 核上。写向量算子应该用 `alloc_ub`。

限制：`alloc_ub` 和 `alloc_shared` 目前只支持 1D~2D，dtype 只支持 fp16/fp32；`alloc_L1` 支持 1~5D。

2.2 搬运与对齐

`T.copy` 支持 GM↔UB、UB↔UB，输入输出 shape 要一致。Expert 模式搬 Cube 数据用 `load_nd2nz`（GM→L1，顺带 ND→NZ）和 `store_fixpipe`（L0C→GM，可顺带 nz2nd 和 relu）；Developer 模式统一用 `copy`。文档表格里 `load_nd2nz` 只列了 fp16 支持，`store_fixpipe` 只列了 fp32 输出。

对齐是这部分最实际的约束。加载指令要求 32B 对齐，尾轴不是整数倍时编译器自动填充：fp16 尾轴取 16 的倍数，fp32 取 8，int8 取 32。文档举例 fp16 尾轴 15 会按 16 加载，多占 6.7%，尾轴为 1 时性能急剧下降。

对齐同时影响片上占用和带宽。UB 名义 192KB，不对齐时能放的有效数据更少，所以 tile 尺寸应当从对齐要求反推，而不是先选一个"整齐"的数。尾块也不是可以留到最后的边界情况，它属于主路径设计。

尾块的写法在 `vec_add_1d.py:31-38`：

```
t0 = cid * block_N
t0 = shape - t0
tail_size = T.min(block_N, t0)
T.copy(A[cid * block_N : cid * block_N + tail_size], A_VEC[0:tail_size])
```

但这段代码实际没有处理非整除形状。`vec_add(N, block_N)` 里 `n_num = N // block_N`（第 18 行），而示例调用是 `vec_add(seq_len, seq_len)`，即 `block_N = N`，所以 `n_num = 1`，`tail_size` 恒等于 `block_N`。如果换成 `N` 不能被 `block_N` 整除的参数，地板除会少下发一个块，尾部元素根本不会被处理；要让这段 tail 逻辑生效，块数应当用 `T.ceildiv(N, block_N)`。

2.3 流水与同步

`T.Pipelined(start, stop=None, num_stages=0, ...)` 的 `num_stages` 默认是 **0，表示不启用流水**（`language/pipeline.py:11-19`，docstring 原文 "if num_stages is 0, pipeline will not be enabled"）。要手工做搬运与计算重叠、双缓冲，必须显式给 `num_stages=2` 之类的值。开发指南建议 K 方向迭代数至少 4~6，否则建立和排空开销占比大，双缓冲还会多占片上内存。

不过 NPU 上多缓冲的主路径不是 `T.Pipelined`，而是编译选项 `--enable-auto-multi-buffer`：`jit/jit_npu.py:1465-1470` 里它默认取 `True`，由后端自动插入双缓冲和同步。也就是说 GEMM 示例里那句 `for k in T.Pipelined(..., num_stages=2)` 是显式写法，不写 `T.Pipelined` 也仍然有双缓冲。

同步有两套，作用范围不同：

核内、pipe 之间：`set_flag(other_pipe, event_id)` 和 `wait_flag(pipe, event_id)`，`event_id` 取 0~15，必须在 `with T.rs("PIPE_XXX")` 内调用 —— 函数内部通过 `ResourceSpecializeFrame.Current()` 取当前 frame 的名字作为 pipe（`customize_npuir.py:1848 / 1861`）。通道枚举为 `PIPE_S / PIPE_V / PIPE_M / PIPE_MTE1~5 / PIPE_FIX`，典型三段式：MTE2 搬入 → 通知 Vector 计算 → Vector 算完通知 MTE3 写回。

跨核、Cube ↔ Vector 之间：`T.sync_block_set(id)` 和 `T.sync_block_wait(id)`。仓库里唯一用例是 `examples/mixcv_mixkernel.py`：Cube 在 `T.rs("PIPE_FIX")` 里 `sync_block_set(0)` 置位，Vector 在 `T.rs("PIPE_MTE2")` 里 `sync_block_wait(0)` 等待。

两个 frame 也不是同一层级：`T.Scope("Cube")` / `T.Scope("Vector")` 选的是**核类型**（`customize_npuir.py:1939`），`T.rs("PIPE_XXX")` 是在**当前核内**选 pipe（`customize_npuir.py:1833`）。API 文档段把 `T.Scope` 当同步作用域举例是错的，示例里的 `T.rs` 才是对的。

通道表实际上就是硬件数据通路的枚举。后面做 profiling 时，“Cube 利用率 / 搬运带宽 / 同步等待”的三分法可以直接落成“哪个 PIPE 忙、哪个 PIPE 在等”，Expert 模式手排同步相当于手动决定流水线重叠的上限。

2.4 开核与分核

`T.Kernel(*blocks, is_npu=True)` 的网格*支持 *1D* 或 *2D*

- `language/kernel.py:251`行：`assert len(blocks) <= 2, "NPU kernel only supports up to 2D blocks"`
- `src/ir.cc:176`：`ICHECK(grid_size.size() <= 2) << "NPU kernel only supports 1D or 2D grid size"`

文档 `docs/Tilelang.language/编译器提示操作/T.Kernel.md:45` 写"必须有且仅有 1 个 block 维度"，还引用了代码里根本不存在的报错 `"NPU kernel must have exactly one block dimension"` —— 这一条是**文档过期**，不是代码限制。

返回值随维度变化（`ir.cc:182-202`）：

维度	返回	`vid` 的 extent
1D	`(cid, vid)`	`grid * 2`
2D	`(bx, by, vid)`	`grid[0] * grid[1] * 2`

`cid` 映射 `blockIdx.x`，`vid` 映射 `blockIdx.z`，是 Vector 子核编号（A2 每个 Cube 配 2 个 Vector）。`mixcv_mixkernel.py` 里把它当 `subid` 用，`subid * (block_M // 2)` 把 `block_M` 对半分给两个向量核。

仓库里没有任何 2D 示例，2D 路径未经实例验证，动手时仍然推荐 1D + 手动拆二维坐标 `cid // n`、`cid % n`。两种分核写法：

```
# 全下发：块数等于 tile 总数
with T.Kernel(T.ceildiv(N, block_N) * T.ceildiv(M, block_M), is_npu=True) as
(cid, _):
    by = cid // T.ceildiv(N, block_N)
    bx = cid % T.ceildiv(N, block_N)

# 固定核数 + 核内串行（vec_add_2d.py）
with T.Kernel(20, is_npu=True) as (cid, _):
    for i in T.serial(T.ceildiv(m_num * n_num, 20)):
        block_id = i * 20 + cid
        if block_id < m_num * n_num:
            ...
```

文档口径是 A2 Cube 约 20~24 个，Vector 核数量翻倍；逻辑核超出物理核会被串行调度。示例里的 `BLOCK_SIZE = 20`，代码注释同时写"实际应通过接口获取"，所以 20 只是示意值，不是常量。

更实用的判据是波次：`ceil(逻辑核数 / 物理核数)`。例如 31 个逻辑核跑在 20 个物理核上需要 2 波，第二波只用 11 个核。分核的目标是波次少且尾波满，不是"接近 20"。

3. 计算原语与成本

向量指令以 `v` 开头：`vadd / vsub / vmul / vdiv / vabs / vsigmoid / vexp / vln / vsqrt` 等，映射到 `hivm::VxxxOp`。操作数必须在 UB；`vadd` 支持广播，规则是 `[M,N]+[M,N]`、`[M,N]+[M,1]`、`[M,N]+scalar`，只允许 `src1` 为标量，`dst` 可以原地。dtype 基本只支持 fp16/fp32；`vabs` 的类型覆盖略有不同（另支持 uint8、int32、uint64），这类不对称反映的是向量单元的实际能力，不是文档疏漏。

`vsigmoid` 的 IR 映射是 `VAdd + VSub + VExp + VDiv` 四条，不是单条指令。DSL 里一个函数对应几条硬件指令，调优时应当按 IR 数，而不是按源码行数估成本。同样，Developer 模式的 `copy` 在跨 dtype 时会插入 `hivm::VCastOp`，类型转换可能藏在搬运里。

`T.gemm`（Developer）和 `T.npuir_dot`（Expert）都是 2D×2D→2D，`size=[a,b,c]` 对应 `[a,b]×[b,c]→[a,c]`。fp16 输入时累加器必须是 fp32，文档明确写写成 fp16 可能运行时卡死；`initC=True` 表示清零，False 表示累加。

`T.Parallel` 用来写标量表达式，支持加减乘除、比较、`T.exp`、`T.sigmoid`、`T.vbrc` 和 `T.if_then_else`。限制是下标不能变换（不能写 `i+1`），操作数必须是 UB buffer，不能直接使用 GM 上的 `T.Tensor` 入参，条件里也不能直接用循环变量。能用 `v*` 表达时优先用 `v*`，因为它直接对应一条 HIVM 指令；超出覆盖范围再用 `T.Parallel`，然后看 IR 确认向量化结果。

规约用 `T.reduce(src, dst, dims, reduce_mode, clear, size)` 或 `reduce_sum/max/min`。约束是 src 与 dst 同 rank，恰好一维不同且 dst 在该维为 1，例如 `(M,N,K)→(M,1,K)` 合法，`(M,N,K)→(M,1,L)` 非法。模式有 sum/max/min/abssum/absmax，映射到 `hivm::VReduceOp`。

4. 从示例看到的调度差异

三个示例是同一类逐元素计算的不同调度：

- `vec_add_1d.py`：块数等于 tile 数，示例参数退化为 1 个块，尾块逻辑未触发（见 2.2）；
- `vec_add_2d.py`：固定 20 个核，核内循环处理多个 tile，演示“固定核数 + 核内串行”，用于大形状时摊薄核启动开销；
- `example_elementwise_add.py`：用 `T.Parallel` 写标量加法，UB staging 后计算再写回。

`vec_add_2d.py` 的 `m_num = M // block_M`、`n_num = N // block_N` 也假设整除。换成非整除形状需要自己改成 ceil 并补尾块。

这三个例子说明，即使是 vec_add，算术部分也只有一行，其余全是分核、尾块、dtype 和 UB staging。算子自动生成的主要难点在调度和布局，不在数学翻译；评测只测整除形状会漏掉尾块、多波次和对齐填充。

附录：API 速查

A.1 开核与调度

API	要点
`T.Kernel(blocks, threads=None, is_npu=True)`	NPU 仅 1 个 block 维；返回 `(cid, vid)`；`threads` 在示例中未使用
`T.serial(n)`	核内串行循环
`T.Scope("Cube")`	Expert 模式圈定 Cube 作用域

A.2 内存分配

API	空间	模式	限制	IR
`T.alloc_ub(shape, dtype)`	UB	Expert	1D~2D；fp16/fp32	`memref::AllocOp`
`T.alloc_shared(shape, dtype)`	UB 或 L1	Developer	1D~2D；fp16/fp32	`tensor.empty`
`T.alloc_L1(shape, dtype)`	L1	Expert	1D~5D；fp16/fp32	`memref::alloc`
`T.alloc_L0C(shape, dtype)`	L0C	Expert	示例为 2D，配 `T.Scope("Cube")`	—
`T.alloc_fragment(shape, dtype)`	片段/累加	两模式示例	无独立文档	—

A.3 搬运

API	方向	模式	限制	IR
`T.copy(src, dst)` / `T.copy(src, dst, size)`	GM↔UB、UB↔UB	两模式	输入输出 shape 一致；支持切片	Expert: `memref::CopyOp`；Developer: tensor 级操作
`T.load_nd2nz(src, dst, size)`	GM→L1	Expert	src 2~4D，dst 2D；文档表仅 fp16；ND→NZ	`hivm::ND2NZOp`
`T.store_fixpipe(src, dst, size, enable_nz2nd, channel_split, pre_relu_mode)`	L0C→GM	Expert	src 2D，dst 2~5D；文档表仅 fp32	`hivm.hir.fixpipe`

A.4 计算

API	要点	IR
`T.vadd / vsub / vmul / vdiv`	逐元素；操作数在 UB；支持广播，仅 `src1` 可为标量；`dst` 可原地	`hivm::VAddOp` 等
`T.vabs(src, dst)`	逐元素绝对值	`hivm::VAbsOp`
`T.vsigmoid(src, dst)`	sigmoid	`VAdd + VSub + VExp + VDiv`
`T.gemm(src1, src2, dst, size=[a,b,c], initC, a_transpose, b_transpose)`	2D×2D→2D；fp16 输入累加器须 fp32；`initC=True` 清零，False 累加	`hivm.hir.mmadL1`
`T.npuir_dot(...)`	`T.gemm` 的 Expert 版本	同上
`T.Parallel(M, N)`	标量表达式并行；下标不可变换；操作数须为 UB buffer；条件不可用循环变量	向量化后进入 vector op

A.5 规约

API	要点	IR
`T.reduce(src, dst, dims, reduce_mode, clear, size)`	src/dst 同 rank；恰好一维不同且 dst 该维为 1；模式 sum/max/min/abssum/absmax	`hivm::VReduceOp`
`T.reduce_sum / reduce_max / reduce_min`	常用模式的便捷封装	同上

A.6 同步与流水

API	要点	IR
`T.set_flag(other_pipe, event_id)`	仅 Expert；`event_id` 0~15；需在作用域内调用	`hivm.hir.set_flag`
`T.wait_flag(pipe, event_id)`	与 `set_flag` 配对；通道 `PIPE_S/V/M/MTE1~5/FIX`	同上
`T.Pipelined(iter, num_stages=2)`	编译器做搬运/计算重叠与双缓冲；迭代数建议 ≥4~6；无独立文档页	—